

OpenClaw Natural Interaction + Multimodal Visual Understanding

OpenClaw Natural Interaction + Multimodal Visual Understanding

1. Course Content
 - Course Overview
 - Learning Objectives
2. Preparation
3. Vision Function Tools
4. Case Demonstration
 - 4.1 Configure WhatsApp Auto Image Sending
5. Source Code Analysis
 - 5.1 `seewhat()` — Capture Camera Image
 - 5.2 `getbbox()` — Get the Target Bounding Box Coordinates
6. FAQ
 - 6.1 What if the vision model frames the target object inaccurately?

1. Course Content

Course Overview

- The OpenClaw multimodal vision feature provides visual perception capabilities through the MCP interface, allowing the robot to "understand" its surroundings
- Covers 3 core vision tools: SeeWhat (capture image), GetBbox (target detection bounding box)
- Supports configuring SeeWhat auto-send channels to automatically push images observed by the robot to communication channels such as Feishu and WhatsApp

Learning Objectives

1. Understand the functions and purposes of the 3 vision control interfaces provided by OpenClaw
2. Master the configuration method of the SeeWhat auto-send channels
3. Master the way to directly call vision tools through the CLI

2. Preparation

```
ros2 launch microros_v2_bringup openclaw_service.launch.py
```



```
robot_control call-tool GetBbox --query The blue block on the table
```

- If the target object is within the camera's field of view, the Box bounding box coordinates and the path to the image with the bounding box will be returned. The Box bounding box coordinates are used by OpenClaw to decide the next operation.

```
yahboom@yahboom:~$ robot_control call-tool GetBbox --query 桌子上的蓝色方块
{
  "result": {
    "bbox": "{ 'x1': 490, 'y1': 287, 'x2': 639, 'y2': 479 }",
    "verify_image": "/home/yahboom/microros_ws/cache/verify_image.png"
  }
}
```

- The automatically saved image path is at:

```
$HOME/microros_ws/cache/verify_image.png
```

- You can check whether the vision model has accurately framed the target through the image with the bounding box here

[!IMPORTANT]

- The accuracy of visual localization bounding box selection depends on the capability of the vision model and will directly affect the accuracy of the visual tracking function. If the vision model frequently frames inaccurately, refer to **6. FAQ** to change the vision model

Configure the SeeWhat auto-send channels

- This function is used to automatically send the image from the observation perspective to the mobile communication channels when the robot calls SeeWhat while observing the environment
- Configuration parameter path: `$HOME/.openc1aw/openc1aw.json`. You can directly modify the configuration file or configure it through CLI commands. The following uses CLI command configuration as an example

[!IMPORTANT]

- SeeWhat_channels are the channels to which images are automatically sent; images will only be sent to the channels in the list

```
...
"robot-control": {
  "enabled": true,
  "config": {
    "seewhat_channels": [
      "whatsapp",
      "feishu",
      "weixin"
    ],
    "feishu_chat_id": "oc_c10ce0aa182cb3f6162d003cxxxxxxx",
    "whatsapp_number": "+8615338xxxxxxx"
  }
}
...
```

4.1 Configure WhatsApp Auto Image Sending

```
openclaw config set plugins.entries.robot-control.config.SeeWhat_channels
'["feishu","weixin","whatsapp"]'
```

- Set the phone number for SeeWhat_channels

```
openclaw config set plugins.entries.robot-control.config.whatsapp_number
"+861533xxxxx"
```

- Restart the gateway for the changes to take effect

```
openclaw gateway restart
```

- When restarting, the OpenClaw log will indicate the auto-send channels

```
15:17:28 [plugins] robot-control: SeeWhat channels loaded: [feishu, weixin, whatsapp]
```

[!NOTE]

- Now you can talk to OpenClaw. When calling the vision function, images will be automatically sent to the configured communication channels

5. Source Code Analysis

- Source Code Path

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

The following explains the source code of the three core vision functions in the `ActionController` class.

5.1 `Seewhat()` — Capture Camera Image

Code Description:

1. **Call the image service:** Calls the `/save_image` service through `image_client` (a ROS2 Trigger client) to trigger the camera to capture
2. **Result handling:** On success, saves the image path to `self.image_path`; on failure, logs the error
3. **Return value:** Returns `[True, image_path]` or `[False, error_message]` for the upper-level function to judge
4. **Image path:** `$HOME/M3Pro_ws/multi_brains_file/image.png`

```
def Seewhat(self)->list[bool,str]:
    '''To capture the scene before your eyes'''
    response:Trigger.Response=self.image_client.call(Trigger.Request())
    if response.success:
        result=[True,response.message]
        self.image_path=response.message
    else:
        self.image_path=None
        result=[False,response.message]
        self.get_logger().error(response.message)
    return result
```

5.2 GetBbox() — Get the Target Bounding Box Coordinates

`GetBbox(query)` function flow:

1. **Auto-refresh the image:** First calls `seewhat()` to get the latest frame; returns an error directly on failure
2. **Draw grid overlay:** Overlays a 30-pixel grid on the image via `draw_grid_overlay(grid_size=30)` to assist the vision model with pixel localization
3. **Call vision detection:** Sends the image with the grid and the target description to the vision model through `self.robot_vision.BoundingBoxDetection(grid_image, query)`
4. **Result verification:** Draws a green rectangle bounding box on the original image and saves it to `verify_image.png` for manual confirmation of whether the framing is accurate
5. **Return coordinates:** Returns `[True, "{x1, y1, x2, y2}"]` or `[False, error_message]`

`BoundingBoxDetection(image_path, target_description)` function flow:

1. **Image encoding:** Reads the image file and converts it to base64 encoding
2. **Call the vision model:** Sends a request through `litellm.completion`, including the system prompt `PROMPT` and the user message (target description + base64 image), using an OpenAI-compatible interface
3. **Error handling:** Catches errors such as API authentication failure, rate limiting, and network exceptions
4. **Parse the returned result:** Extracts the `found, x1, y1, x2, y2` fields from the JSON returned by the model and constructs a `Visual_Result` object to return

```
def GetBbox(self,query:str)->list[bool,str]:
    """Get the bounding box of an object in the scene"""
    res=self.Seewhat() #Automatic image refresh
    if not res[0]:
        return [False, f"Failed to capture image: {res[1]}"]

    img=draw_grid_overlay(image=cv2.imread(self.image_path),grid_size=30)
    cv2.imwrite(self.grid_images_path, img)

    result: Visual_Result =
self.robot_vision.BoundingBoxDetection(self.grid_images_path, query)
    if result.error:
        return [False, result.error_message]
    if not result.found:
        return [False, result.error_message or "Target object not found in
the image."]
```

```

img = cv2.imread(self.image_path)
cv2.rectangle(img, (result.x1, result.y1), (result.x2, result.y2), (0,
255, 0), 2)
cv2.imwrite(self.verify_image_path, img)
return [True, str({"x1": result.x1, "y1": result.y1, "x2": result.x2,
"y2": result.y2})]

def BoundingBoxDetection(self, image_path: str, target_description: str) ->
Visual_Result:
    '''get bounding box of the target object in the image'''
    try:
        with open(image_path, "rb") as f:
            image_base64 = base64.b64encode(f.read()).decode("utf-8")
        except FileNotFoundError:
            return Visual_Result(error=True, error_message=f"Image not found:
{image_path}")

        try:
            response = completion(
                model=self.model,
                messages=[
                    {"role": "system", "content": PROMPT},
                    {
                        "role": "user",
                        "content": [
                            {"type": "text", "text": f"Target:
{target_description}"},
                            {"type": "image_url", "image_url": {"url":
f"data:image/jpeg;base64,{image_base64}"}}],
                    ],
                    api_base=self.base_url,
                    api_key=self.api_key,
                )
            except litellm.AuthenticationError as e:
                return Visual_Result(error=True, error_message=f"Bad API key: {e}")
            except litellm.RateLimitError as e:
                return Visual_Result(error=True, error_message=f"Rate limited: {e}")
            except Exception as e:
                return Visual_Result(error=True, error_message=f"API error: {e}")

            try:
                result_text = response.choices[0].message.content.strip()
                if result_text.startswith("`"):
                    result_text = result_text.split("`")[1]
                    if result_text.startswith("json"):
                        result_text = result_text[4:]
                data = json.loads(result_text)
                if data.get("found"):
                    return Visual_Result(found=True, x1=data["x1"], y1=data["y1"],
x2=data["x2"], y2=data["y2"])
                else:
                    return Visual_Result(found=False, error_message="Target not
found in image")
            except (json.JSONDecodeError, KeyError) as e:
                return Visual_Result(error=True, error_message=f"The visual model
returned an incorrect JSON format. Parse error: {e}, raw: {result_text}")

```

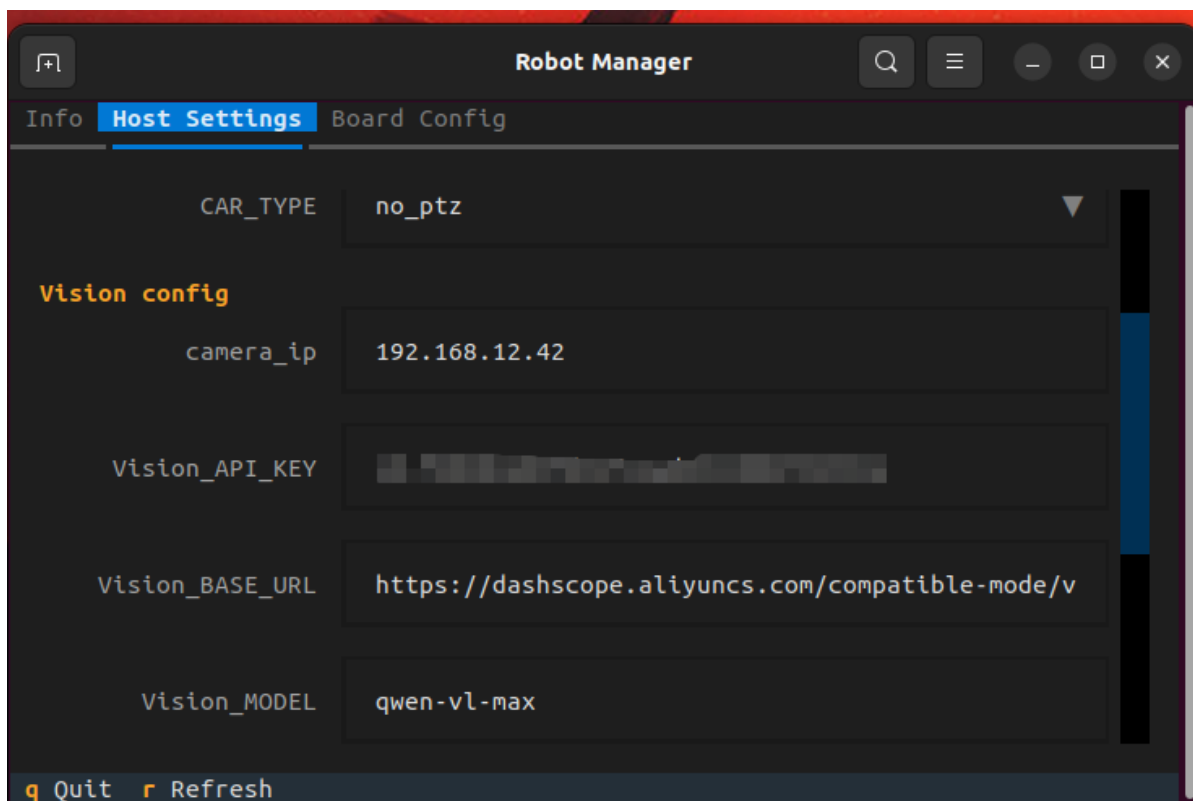
6. FAQ

6.1 What if the vision model frames the target object inaccurately?

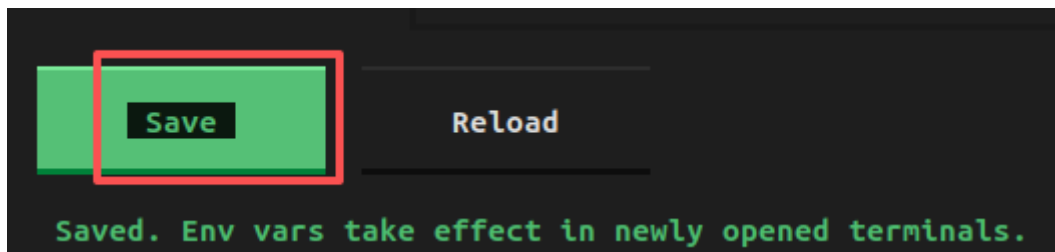
- Visual localization depends on the model's understanding capability. You can change the vision model used. The modification method is as follows
- Double-click the RobotManager application icon on the virtual machine desktop



- On the host settings page, you can modify the vision model provider's base_url, API-KEY, and the corresponding vision model (base_url needs to be compatible with the OpenAI interface protocol)



- After clicking Save, restart the program for the changes to take effect



[!IMPORTANT]

It is recommended to use the Qwenvl-MAX and kimi2.5 series vision models